

An aerial photograph of a city, likely Copenhagen, showing a river, a large stadium (Copenhagens Stadion), and surrounding urban areas. The image is split vertically: the left half shows the actual photograph, and the right half is a solid dark green color.

VFF

1. Semester

DATAANALYSE, ERHVERVSAKADAMIET DANIA

VEJLEDER: Simon A.I.Lersen

GRUPPE 42:

**Hans Hansen
Jens Jensen
Niels Nielsen**

Antal tegn inkl. mellemrum: 99.999.999. Dato: 6. januar 2026

Indholdsfortegnelse

1 Problemstilling	1
1.1 Problemformulering	1
1.2 Evt. af grænsning (osv.... [@sec-pf] hvis du har brug for at referere til asnittet problemformulering)	1
2 Videnskabsteori og metode	1
3 Analyse	1
4 Væsentligste konklusioner	1
5 Literaturliste	2
A Dataforståelse (husk tankegang fra CRISP-DM)	3
B Dataklargørelse	3
B.1 Hente data fra Superstats og gør dem tidy	3
B.2 Hentning af data fra Danmarks Statistik	5
B.3 Hente data fra nager.at og gør dem tidy	6
B.4 Data rensning og tilføjelse af meta-data step 1.	6
B.5 Hente data fra DMI og gør dem tidy	8
B.6 Hente vffkort01 fra Bjarne, load .rds filer	19
B.7 RSQLight	20
B.8 Download jointet datasæt, formatér behørigt, og gør data klar til preprocessing	22
B.9 Preprocess - dan flere variabler og lav ét datasæt uden missing med en række for hver hjemmekamp, og fjern alle variabler, som ikke skal indgå i	23
C Modellering - funktion til at køre lineær regression, best subset selection, ridge/lasso regression og PCR/PLS	25
C.1 Sammenlign på validation de forskellige MSE fra hver model	43
D Evaluering	43
D.1 Hvordan er Test-RMSE sammenlignet med CV-RMSE	43
D.2 Fortokning	44
D.3 Prædiktion på nye data (fx best-case/worst-case scenario for de første kampe i foråret)	44
D.4 Sammenlign resultatet med formålet med undersøgelsen.	45
E Eksempler på markdown-sproget	45
F Fed og kursiv, og punktform	46

Figuroversigt

1	Faserne i CRISP-DM.	45
2	Australsk BNP pr capita	46

Tabeloversigt

1	Outliers og indflydelsesrige observationer.	46
---	---	----

Forslag til afsnit i synopsen:

1 Problemstilling

1.1 Problemformulering

1.2 Evt. af grænsning (osv.... [`@sec-pf`] hvis du har brug for at referere til asnittet problemformulering)

2 Videnskabsteori og metode

3 Analyse

4 Væsentligste konklusioner

5 Literaturliste

Bang, Claus Grand. 2024. *Data-Driven Decision-Making for Business*. 1. udg. Routledge.

VFF. 2025. "Transskriberede interviews". <https://eadania.mrooms.net/mod/hsuforum/discuss.php?d=33332>.

Al kode skal være i bilag (appendix)

Eksempler på afsnit i bilag:

A Dataforståelse (husk tankegang fra CRISP-DM)

B Dataklargørelse

```
fff <- 1+1
```

B.1 Hente data fra Superstats og gør dem tidy

Først installerer vi packages vi skal bruge til projektet og loader dem med `pacman::p_load`.

Derefter skal vi have en variabel som returnerer de år hvor VFF er i superligaen. Den måde vi gør det på, er at vi åbner `superstats.dk`, hvor vi har fundet en tabel der viser de år VFF har været i superligaen. Vi finder så den rigtige tabel, ved at kigge på “inspect element” “table#sortTable”

Nu skal vi have den rigtige string ud fra tabellen, der bruger vi `str_extract`, sletter det der kommer før “/”, og beholder 4 cifre efter /. Disse tal bliver sat ind i en ny variabel vi kalder `sæson_år`. Den returnerer alle årene efter år 2003, hvor VFF var i superligaen.

```
pacman::p_load(tidyverse, vroom, janitor, polite, rjstat, rvest, lubridate,
               stringi, httr, jsonlite, purrr, utils, RSQLite, DBI, leaps,
               glmnet, pls)

# ---- HENT DATA FRA SUPERSTATS ---- #
### Find de år hvor VFF er i superligaen
superliga_url <- "https://superstats.dk/hold/alltime?id=11"
read_superliga <- read_html(superliga_url, encoding = "UTF-8")

# Læs den korrekte table, fundet ved at kigge på HTML siden
# Tilføj kolonne og fjern tekst før / i skemaet
superliga_table_read <- read_superliga %>%
  html_element("table#sortTable") %>%
  html_table(fill = TRUE) %>%
  mutate(
    sæson_år = str_extract(Sæson, "(?<=/)[0-9]{4}") %>% as.integer()
  )

sæson_år <- superliga_table_read %>%
  filter(!is.na(sæson_år) & sæson_år >= 2003) %>%
  pull(sæson_år)

# sæson_år returnerer
# [1] 2026 2025 2024 2023 2022 2017 2016 2014 2008 2007 2006 2005 2004 2003
```

Nu skal vi hente alt data fra superstats. Vi bruger `sæson_år` årstallet og sætter ind i en URL kaldt `sæson_urls`. Nu opretter vi en tibble, hvor alt data skal puttes ind i. Vi har to for loops. Det første for loop trækker alle tables ud for hvert årstal. Den næste for loop bruges til at finde den korrekte information og tage med, vi gør nogle forskellige ting for at sikre det. Først ignorerer vi alle tables som ikke inkluderer “Runde”, dette gøres med en if statement hvor vi bruger `grepl()` til at tjekke om kolonnerne indeholder “Runde”.

Så har vi kode til at tage TV kanaler ud. På superstats siden er der brugt et ikon billede med navnet af kanalen som en titel. Så vi finder billed titlen og trækker den ud. Så bruger vi en if-statement som tjekker om de fundne

TV kanaler matcher antallet af rækker i datasættet, hvis de gør det, tilføjes det som en ny kolonne, ellers sættes værdien til NA.

Vi bruger janitor datapakken til `clean_names()`, derefter navngiver kolonnerne så vi får en konstant datastruktur. Vi ekstraherer så tilskuer variablen som en tekst variabel, fjerner punktum og laver den numerisk.

Runde nummer findes i thead og den ekstraherer vi fordi vi ønsker vi at bruge som en predictor senere. Til sidst bruger vi `bind_rows()` for at få dem ind i den tibble vi startede med at initialisere.

```
min_url <- "https://superstats.dk/program?season="
sæson_urls <- c()
sæson_urls <- paste0(min_url, sæson_år)

kombineret_runde_table <- tibble()
# tryCatch er error handling - sker der en fejl kører den koden i error=function(e) i stedet
tryCatch({
  # tilføjer en pause mellem requests (1) = 1 sek for at undgå overbelastning af serveren
  Sys.sleep(1)

  for (i in seq_along(sæson_urls)) {
    url <- sæson_urls[i]
    page <- read_html(url, encoding = "UTF-8")
    individuel_sæson <- sæson_år[i]

    # Ekstraher alle tables som HTML nodes
    table_nodes <- page %>% html_nodes("table")

    for (tbl_node in table_nodes) {
      df <- html_table(tbl_node, fill = TRUE)

      # Ignorer alle skemaer hvor "Runde" ikke er inkluderet
      if (!any(grepl("Runde", names(df), ignore.case = TRUE))) next

      # TV kanal info
      tv_titles <- tbl_node %>% html_nodes("td img[title]") %>% html_attr("title")
      if (length(tv_titles) == nrow(df)) df$tv <- tv_titles else df$tv <- NA_character_

      df$sæson <- individuel_sæson

      # Clean names & rename, indsæt kolonnenavne
      df <- df %>% clean_names()
      col_names <- c("ugedag", "dato", "kamp", "stilling", "tilskuere", "dommer", "delete", "tv_kanal", "sæson")
      names(df)[1:min(length(col_names), ncol(df))] <- col_names[1:min(length(col_names), ncol(df))]

      # Ekstraher tilskuere som tekst direkte fra HTML
      tilskuere_text <- tbl_node %>%
        html_nodes("td:nth-child(5)") %>%
        html_text(trim = TRUE)

      if (length(tilskuere_text) == nrow(df)) {
        df$tilskuere <- as.numeric(gsub("\\.", "", tilskuere_text))
      }

      # Find runde tekst og indsæt som ny kolonne
      runde_text <- tbl_node %>%
        html_node("thead") %>%
        html_text(trim = TRUE)
```

```

    df$runde <- as.character(stringr::str_extract(runde_text, "Runde\\s*\\d+"))

    # Tilføj til tibble
    kombineret_runde_table <- bind_rows(kombineret_runde_table, df)
  }
}
}, error = function(e) {
  # Denne del kører kun hvis der sker en fejl
  cat(" Fejl ved hentning af data for", year, ":", e$message, "\n")
})

```

B.2 Hentning af data fra Danmarks Statistik

```

# ---- HENT BEFOLKNINGSDATA FRA DANMARKS STATISTIK ---- #
# Definer URLs
urls <- list(
  for_2005 = "https://api.statbank.dk/v1/data/bef1a/JSONSTAT?OMRÅDE=791%2C761%2C763%2C769%2C775%2C789&Tid=2005%2C2006%2C2007",
  år_2005_2007 = "https://api.statbank.dk/v1/data/bef1a07/JSONSTAT?OMRÅDE=791&Tid=2005%2C2006%2C2007",
  efter_2008 = "https://api.statbank.dk/v1/data/folk1a/JSONSTAT?OMRÅDE=791&KØN=TOT&ALDER=IALT&CIVILSTAN=TOT"
)

# Hent og bearbejd data for 2003-2004 (skal summeres da Kommunen var opdelt anderledes den gang)
stat_viborg_for_2005 <- fromJSONstat(urls$for_2005) %>%
  as_tibble() %>%
  pull(1) %>%
  group_by(tid) %>%
  summarise(value = sum(value, na.rm = TRUE)) %>%
  mutate(
    år = as.character(tid),
    kvartal = as.character(NA)
  ) %>%
  dplyr::select(år, kvartal, value)

# Hent og bearbejd data for 2005-2007
stat_viborg_2005_2007 <- fromJSONstat(urls$år_2005_2007) %>%
  as_tibble() %>%
  pull(1) %>%
  dplyr::select(tid, value) %>%
  mutate(
    år = as.character(tid),
    kvartal = as.character(NA)
  ) %>%
  dplyr::select(år, kvartal, value)

# Hent og bearbejd data efter 2008 (nu med kvartals optælling)
stat_viborg_after_2008 <- fromJSONstat(urls$efter_2008) %>%
  as_tibble() %>%
  pull(1) %>%
  dplyr::select(tid, value) %>%
  mutate(
    år = str_extract(tid, "^\\d{4}"),
    kvartal = str_extract(tid, "K\\d")
  ) %>%

```

```

dplyr::select(år, kvartal, value)

# Kombiner datasæt til én samlet tibble
viborg_befolkning_komplet <- bind_rows(
  stat_viborg_for_2005,
  stat_viborg_2005_2007,
  stat_viborg_efter_2008
) %>%
  arrange(år, kvartal) %>%
  rename(Indbyggere_Viborg_Kommune = value)

```

B.3 Hente data fra nager.at og gør dem tidy

Data hentes fra nager.at ved at indsætte sæson_år fra tidligere, så får vi de år hvor VFF er i superligaen.

```

# ---- HENT DATA FRA DATE.NAGER.AT ---- #
alle_helligdage <- tibble()
for (year in sæson_år) {
  url <- paste0("https://date.nager.at/api/v3/PublicHolidays/", year, "/DK")
  response <- GET(url)
  helligdage <- fromJSON(content(response, "text", encoding = "UTF-8")) %>%
    select(date, localName)
  alle_helligdage <- bind_rows(alle_helligdage, helligdage)
  Sys.sleep(1)
}

```

B.4 Data rensning og tilføjelse af meta-data step 1.

```

# ---- DATARENSNING ---- #
vff_kampdata_upload <- kombineret_runde_table %>%

# --- Split dato/tid i separate kolonner ---
separate(dato, into = c("dato", "tid"), sep = " ", remove = FALSE) %>%

# --- Identificer VFF-kampe ---
mutate(
  vff_spiller = grepl("(VFF|Viborg)", kamp, ignore.case = TRUE)
) %>%
filter(vff_spiller, !is.na(tilskuere)) %>%

# --- Sæson & år ---
mutate(
  sæson = paste0(sæson - 1, "/", sæson), # vis sæson korrekt
  month = as.numeric(str_sub(dato, 4, 5)),
  end_year = as.numeric(str_sub(sæson, 6, 9)),
  # Tildel korrekt kalenderår (sæson starter i juli, slutter i maj)
  år = ifelse(month >= 7 & month <= 12, end_year - 1, end_year)
) %>%

# --- Dato ---
mutate(kamp_dato = dmy(paste0(dato, "/", år))) %>%

```

```

# --- Tid & tidsgrupper ---
# Tilføj tids kolonne, tidligt midt sent
mutate(
  # Formater tid så de står i timer og minutter (HH:MM)
  tid = na_if(tid, ""), # hvis NA eksisterer brug "" i stedet
  klokkeslæt = hms::parse_hm(tid),
  tidsgruppe = case_when(
    klokkeslæt >= hms::parse_hm("12:00") & klokkeslæt < hms::parse_hm("15:30") ~ "tidligt",
    klokkeslæt >= hms::parse_hm("15:30") & klokkeslæt < hms::parse_hm("18:30") ~ "midt",
    klokkeslæt >= hms::parse_hm("18:30") & klokkeslæt <= hms::parse_hm("23:59") ~ "sent"
  )
) %>%

# --- Score & resultat ---
# Tilføj kolonne hvor VFF har vundet sidste kamp
separate(stilling, into = c("score_home", "score_away"), sep = "-", convert = TRUE, remove = FALSE) %

# Tilføj kolonne med VFF score og modstander score, og navne
mutate(
  vff_score = ifelse(grepl("^(VFF|Viborg)", kamp), score_home, score_away),
  modstander_score = ifelse(grepl("^(VFF|Viborg)", kamp), score_away, score_home),
  vff_vundet = case_when(
    vff_score > modstander_score ~ "vundet",
    vff_score < modstander_score ~ "tabt",
    TRUE ~ "uafgjort"
  )
) %>%

# Identificer om VFF spillede hjemme (kampnavn starter med VFF/Viborg)
mutate(
  vff_hjemme = grepl("^(VFF|Viborg)", kamp, ignore.case = TRUE)
) %>%

#Behold hjemmekampe
filter(vff_hjemme == TRUE) %>%

# --- Sortér kronologisk ---
arrange(kamp_dato) %>%

# --- Historiske resultater ---
mutate(
  seneste_kamp = lag(vff_vundet), # Resultat af forrige hjemmekamp
  vff_vundet_2 = lag(vff_vundet, 1) == "vundet" &
    lag(vff_vundet, 2) == "vundet" # 2 sejre i træk før denne kamp
) %>%

# --- Variabelvalg ---
# Behold relevante kolonner til analyse
dplyr::select(
  sæson, kamp_dato, ugedag, tid, runde, kamp, stilling, tilskuere, dommer,
  tv_kanal, tidsgruppe, seneste_kamp, vff_vundet_2, år
)

# ---- Integrer API data ind i superstatsdata i RStudio (Vi gør det også i SQL) ----
vff_kampdata <- vff_kampdata_upload %>%

```

```

# --- Helligdage ---
# Indsættelse af helligdag data fra date.nager.at og ser om datoen matcher kamp_dato
mutate(helligdag = kamp_dato %in% as.Date(alle_helligdage$date)) %>%
# --- Befolkningsdata ---
# join med befolkningsdata
mutate(
  år_char = as.character(år),
  kvartal = paste0("K", quarter(kamp_dato))
) %>%
filter(!is.na(kamp_dato)) %>%

# Join kvartalsbaseret befolkningsdata
left_join(viborg_befolkning_komplet, by = c("år_char" = "år", "kvartal")) %>%
# Join årlig befolkningsdata som backup
left_join(
  viborg_befolkning_komplet %>%
    filter(is.na(kvartal)) %>%
    select(år, Indbyggere_Viborg_Kommune) %>%
    rename(Indbyggere_årlig = Indbyggere_Viborg_Kommune),
  by = c("år_char" = "år")
) %>%
# Fyld manglende kvartalsdata med årsdata
mutate(
  Indbyggere_Viborg_Kommune = coalesce(Indbyggere_Viborg_Kommune, Indbyggere_årlig)
) %>%

# Akkumuleret befolkning
# Beregn vækst i indbyggertal
arrange(kamp_dato) %>%
mutate(
  basis_befolkning = first(Indbyggere_Viborg_Kommune), #baseline
  akk_indbyggertal = Indbyggere_Viborg_Kommune - basis_befolkning #vækst
) %>%
# --- Variabelvalg ---
# Behold relevante kolonner til analyse
dplyr::select(
  sæson, kamp_dato, ugedag, tid, runde, kamp, stilling, tilskuere, dommer,
  tv_kanal, helligdag, kvartal, Indbyggere_Viborg_Kommune,
  akk_indbyggertal, tidsgruppe, seneste_kamp, vff_vundet_2
)

```

B.5 Hente data fra DMI og gør dem tidy

```

# Gem RDS
saveRDS(vff_kampdata, "data/vff_kampdata.rds")

# Load RDS
vff_kampdata <- readRDS("data/vff_kampdata.rds")

# ---- SE RESULTAT ---- #
view(vff_kampdata)

# ---- HENT DMI DATA ---- #

```

```

# 1. Læs kampdatoer
datoer <- as.Date(vff_kampdata$kamp_dato, format = "%d-%m-%Y")

# 2. API-parametre
base_url <- "https://dmigw.govcloud.dk/v2/metObs/collections/observation/items?"
stationId <- "stationId=06065" # års Syd
readRenviron("~/Renviron")
apikey <- paste0("api-key=", Sys.getenv("DMI_API_KEY"))

# 3. Samlet dataframe
all_data <- data.frame()

extract_properties <- function(f) {
  # Case 1: GeoJSON format
  if (is.list(f) && "properties" %in% names(f) && is.list(f$properties)) {
    return(as.data.frame(f$properties))
  }

  # Case 2: Already a flat list
  if (is.list(f) && !"geometry" %in% names(f)) {
    return(as.data.frame(f))
  }

  return(NULL)
}

# 4. Hent data for hver kampdato (7 dage før)
for (date in datoer) {
  kamp_dato <- as.Date(date)
  start_date <- kamp_dato - 7
  end_date <- kamp_dato

  datetime <- paste0("&datetime=", start_date, "T00:00:00Z/", end_date, "T23:59:59Z")
  url <- paste0(base_url, stationId, "&", datetime, "&", apikey)

  message("Henter data fra: ", url)

  res <- GET(URLEncode(url))
  json_txt <- rawToChar(res$content)

  if (nchar(json_txt) > 0) {
    data <- fromJSON(json_txt)

    if (!is.null(data$features) && length(data$features) > 0) {
      df <- map_df(data$features, extract_properties)
      df$kamp_dato <- kamp_dato

      all_data <- bind_rows(all_data, df)
    }
  }
}

```

Henter data fra: <https://dmigw.govcloud.dk/v2/metObs/collections/observation/items?stationId=06065&&datetime=2023-01-01T00:00:00Z/2023-01-07T23:59:59Z>

[illegible]

```
# Tjek om den har hentet data
names(all_data)
```

```
[1] "type"          "coordinates"  "parameterId"  "created"      "value"
[6] "observed"      "stationId"    "kamp_dato"
```

```
# 5. Klargør data til wide format
wide_data <- all_data %>%
  select(kamp_dato, observed, parameterId, value) %>%
  mutate(observed = as.Date(observed)) %>%
  filter(parameterId %in% c("precip_past1h", "temp_mean_past1h", "wind_gust_always_past1h")) %>%
  group_by(kamp_dato, observed, parameterId) %>%
  summarise(value = mean(value, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = parameterId, values_from = value) %>%
  rename(regn = precip_past1h, temperatur = temp_mean_past1h, vind = wind_gust_always_past1h) %>%
  arrange(kamp_dato, observed)
```

B.6 Hente vffkort01 fra Bjarne, load .rds filer

```
#### gem og load RDS
saveRDS(vff_kampdata_upload, "data/vff_kampdata_upload.rds")
saveRDS(alle_helligdage, "data/alle_helligdage.rds")
saveRDS(viborg_befolkning_komplet, "data/viborg_befolkning_komplet.rds")
saveRDS(wide_data, "data/wide_data.rds")

vff_kampdata_upload <- readRDS("data/vff_kampdata_upload.rds")
alle_helligdage <- readRDS("data/alle_helligdage.rds")
viborg_befolkning_komplet <- readRDS("data/viborg_befolkning_komplet.rds")
wide_data <- readRDS("data/wide_data.rds")
vffkort01 <- readRDS("data/vffkort01.rds")

# Inspekt data
glimpse(wide_data)
```

```
Rows: 216
Columns: 5
$ kamp_dato    <date> 2002-10-20, 2002-10-26, 2002-11-09, 2002-11-24, 2003-03-26~
$ observed     <date> 2002-10-20, 2002-10-26, 2002-11-09, 2002-11-24, 2003-03-26~
$ regn         <dbl> 0,000000000, 0,009090909, 0,050000000, 0,133333333, 0,00000~
$ temperatur   <dbl> 3,4416667, 8,4909091, 3,6583333, 5,5583333, 9,4250000, 7,95~
$ vind         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 6,1~
```

```
glimpse(viborg_befolkning_komplet)
```

```
Rows: 50
Columns: 3
$ år           <chr> "2002", "2003", "2004", "2005", "2006", "200~
$ kvartal      <chr> NA, NA, NA, NA, NA, NA, "K1", "K2", "K3", "K~
$ Indbyggere_Viborg_Kommune <int> 87621, 88039, 88354, 89645, 90518, 91405, 92~
```

```
glimpse(alle_helligdage)
```

```
Rows: 207
Columns: 2
$ date         <chr> "2026-01-01", "2026-04-02", "2026-04-03", "2026-04-05", "202~
$ localName    <chr> "Nytårsdag", "Skærtorsdag", "Langfredag", "Påskedag", "2. På~
```

```
glimpse(vff_kampdata_upload)
```

```
Rows: 220
Columns: 14
$ sæson        <chr> "2002/2003", "2002/2003", "2002/2003", "2002/2003", "2002~
$ kamp_dato    <date> 2002-08-11, 2002-08-25, 2002-09-01, 2002-09-15, 2002-09-~
$ ugedag       <chr> "Søn", "Søn", "Søn", "Søn", "Søn", "Søn", "Lør", "Lør", "~
$ tid          <chr> "15:00", "17:35", "15:00", "15:00", "15:00", "15:00", "16~
$ runde        <chr> "Runde 3", "Runde 5", "Runde 6", "Runde 8", "Runde 9", "R~
$ kamp         <chr> "VFF-FCM", "VFF-AaB", "VFF- AB", "VFF-EFB", "VFF-KBK", "V~
$ stilling     <chr> "1-1", "2-2", "2-1", "6-0", "1-4", "0-0", "0-3", "1-3", "~
$ tilskuere    <dbl> 3006, 3379, 2443, 3274, 3062, 2594, 8216, 1908, 2018, 555~
```

```

$ dommer      <chr> "Claus Bo Larsen", "Steen Pedersen", "Anders Mosegaard", ~
$ tv_kanal    <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ tidsgruppe  <chr> "tidligt", "midt", "tidligt", "tidligt", "tidligt", "tidl~
$ seneste_kamp <chr> NA, "uafgjort", "uafgjort", "vundet", "vundet", "tabt", "~
$ vff_vundet_2 <lgl> NA, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, FALSE~
$ år          <dbl> 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2002, 200~

```

```
glimpse(vffkort01)
```

```

Rows: 259
Columns: 8
$ sæson      <chr> "2000/2001", "2000/2001", "2000/2001", "2000/2001", "200~
$ runde      <dbl> 2, 4, 5, 8, 9, 11, 13, 15, 17, 19, 20, 22, 24, 26, 28, 3~
$ tilskuere  <dbl> 2089, 3198, 2378, 3878, 2017, 2308, 4132, 2308, 2308, 21~
$ år         <dbl> 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, 20~
$ hold       <chr> "VFF- AB", "VFF-FCK", "VFF-SJF", "VFF-FCM", "VFF-SIF", "~
$ d10_tilskuere <dbl> 1087, 1346, 1114, 1871, 949, 1026, 2181, 1027, 1278, 953~
$ d7_tilskuere <dbl> 1282, 2147, 1607, 2517, 1103, 1451, 2810, 1543, 1437, 10~
$ d3_tilskuere <dbl> 1898, 2735, 2166, 3278, 1769, 1994, 3992, 2049, 1882, 17~

```

```

# Giv vffkort01 kamp_dato variable
vffkort01 <- vffkort01 %>%
  left_join(
    vff_kampdata_upload %>%
      select(år, tilskuere, kamp_dato),
    by = c("år", "tilskuere")
  )
view(vffkort01)

# Ændre format i viborg_befolkning_komplet og alle_helligdage
viborg_befolkning_komplet$år <- as.double(viborg_befolkning_komplet$år)
alle_helligdage$date <- as.Date(alle_helligdage$date)

# Giv viborg_befolkning_komplet kamp_dato variable
viborg_befolkning_komplet <- viborg_befolkning_komplet %>%
  select(år, Indbyggere_Viborg_Kommune) %>%
  group_by(år) %>%
  slice_max(Indbyggere_Viborg_Kommune, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  left_join(
    vff_kampdata_upload %>% select(år, kamp_dato),
    by = "år"
  )

```

B.7 RSQLight

```

# Join i SQLite
con <- dbConnect(RSQLite::SQLite(), "data/VFF_data.sqlite")

uploadtables <- list(
  vff_kampdata_upload = vff_kampdata_upload,
  alle_helligdage = alle_helligdage,
  viborg_befolkning_komplet = viborg_befolkning_komplet,

```

```

wide_data = wide_data,
vffkort01 = vffkort01
)

for(name in names(uploadtables)) {
  cat("Uploading table:", name, "\n")
  dbWriteTable(con, name, uploadtables[[name]], overwrite = TRUE, row.names = FALSE)
}

```

```

Uploading table: vff_kampdata_upload
Uploading table: alle_helligdage
Uploading table: viborg_befolkning_komplet
Uploading table: wide_data
Uploading table: vffkort01

```

```
dbListTables(con, schema = "dbo")
```

```

[1] "alle_helligdage"      "vff_kampdata_upload"
[3] "vffkort01"           "viborg_befolkning_komplet"
[5] "wide_data"

```

```
# dbExecute(con, "DROP TABLE kombineret_runde_table")
```

```
# Your SQL query as a string
```

```

query <- "SELECT
    v.*,
    w.regn,
    w.temperatur,
    w.vind,
    h.localName AS helligdag,
    b.Indbyggere_Viborg_Kommune,
    k.d10_tilskuere,
    k.d7_tilskuere,
    k.d3_tilskuere
FROM vff_kampdata_upload AS v

-- Join wide_data på kamp_dato
LEFT JOIN wide_data AS w
    ON v.kamp_dato = w.kamp_dato

-- Join alle_helligdage på date
LEFT JOIN alle_helligdage AS h
    ON v.kamp_dato = h.date

-- Join viborg_befolkning_komplet på år
LEFT JOIN viborg_befolkning_komplet AS b
    ON v.kamp_dato = b.kamp_dato

-- Join vffkort01 på kamp_dato
LEFT JOIN vffkort01 AS k
    ON v.kamp_dato = k.kamp_dato

-- WHERE fjerner værdier under 0
WHERE v.tilskuere > 0

```

```

-- Hvis flere rækker kamp_dato, behold kun 1
GROUP BY v.kamp_dato

-- Behold grupper hvor der findes 1 kamp på den dato
HAVING COUNT(v.kamp_dato) = 1;
"

# Run the query and get the result as a data frame in R
joined_data <- dbGetQuery(con, query) %>%
  mutate(kamp_dato = as.Date(kamp_dato))

saveRDS(joined_data, "data/joined_data.rds")

dbDisconnect(con)

```

B.8 Download jointet datasæt, formatér behørigt, og gør data klar til preprocessing

```

# Indlæs data
joined_data <- readRDS("data/joined_data.rds")

# Rens og transformer data
joined_data <- joined_data %>%
  mutate(
    # Ugedag som ordnet faktor
    ugedag = factor(ugedag,
                    levels = c("Man", "Tir", "Ons", "Tor", "Fre", "Lør", "Søn"),
                    ordered = TRUE),

    # Tidsgruppe som faktor
    tidsgruppe = factor(tidsgruppe, levels = c("tidligt", "midt", "sent")),

    # Seneste kamp som faktor
    seneste_kamp = factor(seneste_kamp, levels = c("vundet", "uafgjort", "tabt")),

    # Regn grupperet i kategorier
    regn_gruppe = case_when(
      is.na(regn) ~ NA_character_,
      regn == 0 ~ "ingen regn",
      regn > 0 & regn <= 1 ~ "lidt regn",
      regn > 1 ~ "meget regn"
    ) %>% factor(levels = c("ingen regn", "lidt regn", "meget regn")),

    # Helligdag flag
    helligdag = ifelse(is.na(helligdag), 0, 1),

    # Runde som numerisk
    runde = as.numeric(gsub("Runde ", "", runde))
  ) %>%
  # Beregn akkumuleret befolkningsvækst
  mutate(
    akk_indbyggertal = Indbyggere_Viborg_Kommune - first(Indbyggere_Viborg_Kommune)
  )

```

```

# Tilføj kamp_gruppe kolonne baseret på gennemsnitlige tilskuere for hver matchup
joined_data <- joined_data %>%
  group_by(kamp) %>%
  mutate(
    avg_tilskuere = mean(tilskuere, na.rm = TRUE),
    kamp_gruppe = case_when(
      avg_tilskuere <= 4000 ~ "lille",
      avg_tilskuere <= 5500 ~ "middel",
      TRUE ~ "stor"
    ) %>% factor(levels = c("lille", "middel", "stor"))
  ) %>%
  ungroup() %>%
  select(-avg_tilskuere) # fjern hjælpekolonne

# Flag helligdage og skoleferier
joined_data <- joined_data %>%
  mutate(
    uge = isoweek(kamp_dato),
    ferie_navn = case_when(
      uge == 7 ~ "vinterferie",
      uge %in% 26:32 ~ "sommerferie",
      uge == 42 ~ "efterårsferie",
      uge %in% 52:1 & month(kamp_dato) %in% c(12, 1) ~ "juleferie",
      helligdag == 1 ~ "helligdag",
      TRUE ~ NA_character_
    ),

    ferie_flag = factor(
      ifelse(!is.na(ferie_navn), 1, 0),
      levels = c(0, 1),
      labels = c("nej", "ja")
    )
  )

# Tilføj salg_3, salg_7, salg_10 kolonner ud fra d3_tilskuere, d7_tilskuere, d10_tilskuere til brug i M
joined_data <- joined_data %>%
  mutate(
    salg_10 = d10_tilskuere, # billetter solgt 10 dage før
    salg_7 = d7_tilskuere - d10_tilskuere, # yderligere billetter solgt mellem dag 10 og 7
    salg_3 = d3_tilskuere - d7_tilskuere, # yderligere billetter solgt mellem dag 7 og 3
  )

# Gem rensset data
saveRDS(joined_data, "data/joined_data_clean.rds")

```

B.9 Preprocess - dan flere variabler og lav ét datasæt uden missing med en række for hver hjemmekamp, og fjern alle variabler, som ikke skal indgå i

```

# Indlæs data fra data_cleaning.R script
data_clean <- readRDS("data/joined_data_clean.rds")
options(scipen = 999)

# Variabelvalg til machine learning modeller

```

```

# Før 10 dage før kampen (ingen billetsalg endnu)
vff_variabler <- data_clean %>%
  select(tilskuere, runde, år, ugedag, tidsgruppe,
         seneste_kamp, vff_vundet_2, regn_gruppe,
         temperatur, vind, akk_indbyggertal, kamp_gruppe, ferie_flag) %>% na.omit()

# 10 dage før kampen
vff_10 <- data_clean %>%
  select(tilskuere, salg_10, runde, år, ugedag, tidsgruppe,
         seneste_kamp, vff_vundet_2, akk_indbyggertal, kamp_gruppe, ferie_flag) %>% na.omit()

# 7 dage før kampen
vff_7 <- data_clean %>%
  select(tilskuere, salg_10, salg_7, runde, år, ugedag, tidsgruppe,
         seneste_kamp, vff_vundet_2, akk_indbyggertal, kamp_gruppe, ferie_flag) %>% na.omit()

# 3 dage før kampen
vff_3 <- data_clean %>%
  select(tilskuere, salg_10, salg_7, salg_3, runde, år, ugedag, tidsgruppe,
         seneste_kamp, vff_vundet_2, akk_indbyggertal, kamp_gruppe, ferie_flag) %>% na.omit()

#Inspekt
glimpse(vff_variabler)

```

```

Rows: 195
Columns: 13
$ tilskuere      <dbl> 5705, 5349, 3204, 4310, 3092, 4980, 2933, 4419, 3371,~
$ runde          <dbl> 8, 11, 12, 14, 17, 19, 21, 24, 26, 27, 29, 31, 32, 3,~
$ år             <dbl> 2003, 2003, 2003, 2003, 2003, 2003, 2004, 2004, 2004,~
$ ugedag         <ord> Ons, Søn, Søn, Søn, Søn, Søn, Søn, Søn, Søn, Lør, Søn~
$ tidsgruppe     <fct> sent, tidligt, tidligt, tidligt, tidligt, tidligt, ti~
$ seneste_kamp   <fct> vundet, uafgjort, tabt, tabt, vundet, uafgjort, tabt,~
$ vff_vundet_2   <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,~
$ regn_gruppe    <fct> ingen regn, lidt regn, lidt regn, ingen regn, ingen r~
$ temperatur     <dbl> 16,7333333, 8,2000000, 1,7888889, -0,2888889, 5,82222~
$ vind           <dbl> 6,177778, 7,144444, 9,044444, 3,022222, 10,077778, 9,~
$ akk_indbyggertal <int> 418, 418, 418, 418, 418, 418, 733, 733, 733, 733, 733~
$ kamp_gruppe    <fct> stor, middel, lille, stor, lille, stor, middel, stor,~
$ ferie_flag     <fct> nej, nej, ja, nej, nej, nej, nej, nej, nej, nej, nej,~

```

```
glimpse(vff_10)
```

```

Rows: 213
Columns: 11
$ tilskuere      <dbl> 3379, 2443, 3274, 3062, 2594, 8216, 1908, 2018, 5553,~
$ salg_10        <dbl> 1222, 1001, 1330, 998, 1172, 3785, 823, 987, 2292, 22~
$ runde          <dbl> 5, 6, 8, 9, 12, 13, 15, 17, 20, 22, 23, 25, 27, 30, 3~
$ år             <dbl> 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2003,~
$ ugedag         <ord> Søn, Søn, Søn, Søn, Søn, Lør, Lør, Søn, Ons, Søn, Tor~
$ tidsgruppe     <fct> midt, tidligt, tidligt, tidligt, tidligt, midt, midt,~
$ seneste_kamp   <fct> uafgjort, uafgjort, vundet, vundet, tabt, uafgjort, t~
$ vff_vundet_2   <int> 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ akk_indbyggertal <int> 0, 0, 0, 0, 0, 0, 0, 0, 418, 418, 418, 418, 418, 418,~
$ kamp_gruppe    <fct> middel, lille, lille, lille, middel, stor, middel, li~
$ ferie_flag     <fct> nej, nej, nej, nej, ja, nej, nej, nej, nej, nej, ja, ~

```

```
glimpse(vff_7)
```

```
Rows: 213
Columns: 12
$ tilskuere      <dbl> 3379, 2443, 3274, 3062, 2594, 8216, 1908, 2018, 5553, ~
$ salg_10        <dbl> 1222, 1001, 1330, 998, 1172, 3785, 823, 987, 2292, 22~
$ salg_7         <dbl> 893, 488, 712, 1056, 477, 794, 381, 357, 1225, 1677, ~
$ runde          <dbl> 5, 6, 8, 9, 12, 13, 15, 17, 20, 22, 23, 25, 27, 30, 3~
$ år             <dbl> 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2003, ~
$ ugedag         <ord> Søn, Søn, Søn, Søn, Søn, Søn, Lør, Lør, Søn, Ons, Søn, Tor~
$ tidsgruppe     <fct> midt, tidligt, tidligt, tidligt, tidligt, midt, midt, ~
$ seneste_kamp   <fct> uafgjort, uafgjort, vundet, vundet, tabt, uafgjort, t~
$ vff_vundet_2   <int> 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ akk_indbyggertal <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 418, 418, 418, 418, 418, 418, ~
$ kamp_gruppe    <fct> middel, lille, lille, lille, middel, stor, middel, li~
$ ferie_flag     <fct> nej, nej, nej, nej, ja, nej, nej, nej, nej, nej, ja, ~
```

```
glimpse(vff_3)
```

```
Rows: 213
Columns: 13
$ tilskuere      <dbl> 3379, 2443, 3274, 3062, 2594, 8216, 1908, 2018, 5553, ~
$ salg_10        <dbl> 1222, 1001, 1330, 998, 1172, 3785, 823, 987, 2292, 22~
$ salg_7         <dbl> 893, 488, 712, 1056, 477, 794, 381, 357, 1225, 1677, ~
$ salg_3         <dbl> 880, 666, 796, 724, 438, 2713, 328, 424, 1256, 1319, ~
$ runde          <dbl> 5, 6, 8, 9, 12, 13, 15, 17, 20, 22, 23, 25, 27, 30, 3~
$ år             <dbl> 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2002, 2003, ~
$ ugedag         <ord> Søn, Søn, Søn, Søn, Søn, Søn, Lør, Lør, Søn, Ons, Søn, Tor~
$ tidsgruppe     <fct> midt, tidligt, tidligt, tidligt, tidligt, midt, midt, ~
$ seneste_kamp   <fct> uafgjort, uafgjort, vundet, vundet, tabt, uafgjort, t~
$ vff_vundet_2   <int> 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ akk_indbyggertal <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 418, 418, 418, 418, 418, 418, ~
$ kamp_gruppe    <fct> middel, lille, lille, lille, middel, stor, middel, li~
$ ferie_flag     <fct> nej, nej, nej, nej, ja, nej, nej, nej, nej, nej, ja, ~
```

C Modellering - funktion til at køre lineær regression, best subset selection, ridge/lasso regression og PCR/PLS

```
# Funktion til predict i best subset selection
# Predictors bliver lavet om til tal som er klar til matrix multiplikation
# Orthogonal factor som ugedag laves om til ugedagL, ugedagQ, ugedagC osv.
predict.regsubsets <- function(object, newdata, id, formula) {
  mat <- model.matrix(formula, newdata)
  # Fordi regsubsets ikke har en predict() funktion kan vi udregne
  # de bedste koefficienter med "id" predictors
  coefi <- coef(object, id = id)
  # Vi får navnene af predictors (kolonnerne)
  vars <- names(coefi)
  # Her laves matrix multiplikation. Vi ganger predictors i tal-format
  # med koefficienterne
  mat[, vars, drop = FALSE] %*% coefi
}
```

```

}

# Funktion til at køre alle machine learning modeller
run_all_models <- function(data, response, train_frac = 2/3, seed = 8) {
  # Vi definerer en funktion, som skal køre ML modellerne.
  # data = vores udvalgte variabler, response = variabelen vi ønsker at predict,
  # train_frac = 2/3, vi bruger 2/3 af vores rows til predictions.
  # seed = 8, seed inkluderes i funktionen så hvis man ønsker at køre
  # funktionen med en anden seed, kan man tilføje det når man vil se resultaterne.
  # For eksempel results_10 <- run_all_models(vff_10, "tilskuere", seed = 69420)

  set.seed(seed)
  n <- nrow(data)
  # Her splittes tilfældige rows til training. Den ganger 2/3 med totale rows.
  train_idx <- sample(1:n, floor(train_frac * n))
  # Den sidste 1/3 som bliver brugt til tests, fundet med setdiff() funktion
  test_idx <- setdiff(1:n, train_idx)
  train <- data[train_idx, ]
  test <- data[test_idx, ] # Test og training subsets vælges

  # Vi ekstraherer det tal vi ønsker at finde, i vores tilfælde tilskuere
  y_train <- train[[response]]
  y_test <- test[[response]]

  # Laver en ligning der sætter tilskuere mod alle andre predictors
  formula <- as.formula(paste(response, "~ ."))

  # ----- Lineær Regression (alle variabler) -----
  lm.fit <- lm(formula, data = train)
  lm.pred <- predict(lm.fit, newdata = test)
  lm_rmse <- sqrt(mean((lm.pred - y_test)^2))

  # ----- Best Subset Selection -----
  k <- 10
  # sample() tildeler tilfældigt hver training row til et af de 10 folds.
  folds <- sample(rep(1:k, length.out = nrow(train)))
  # cv.errors = tom matrix til at gemme MSE for hver fold og subset størrelse,
  # bliver fyldt op i for loop
  cv.errors <- matrix(NA, k, ncol(train) - 1)

  # Loop over hver fold. regsubsets() finder de
  # bedste modeller med 1 til p predictors.
  for (j in 1:k) {
    # Vi vælger alle rækker der ikke tilhører fold j,
    # så der trænes på 9 folds eller 9/10 af data
    best.fit.cv <- regsubsets(formula, data = train[folds != j, ], nvmax = ncol(train) - 1)

    # Loop over hver subset størrelse inde i hver fold.
    # subset størrelse kan f.eks. være subset 4 = model med 4 kolonner/predictors.
    for (i in 1:(ncol(train) - 1)) {
      # predict.regsubsets(...) laver prediktioner på fold "j" (test fold),
      # med subset størrelsen "i"
      # mean(...) beregner MSE for hver fold "j" og subset størrelse "i".
      # cv.errors får tildelt MSE
      # Dette gøres for at finde den subset model der har lavest fejl.
    }
  }
}

```

```

    pred <- predict.regsbsets(best.fit.cv, train[folds == j, ], id = i, formula = formula)
    cv.errors[j, i] <- mean((train[[response]][folds == j] - pred)^2)
  }
}

# colMeans() giver gennemsnittet af hver kolonne MSE på tværs af alle 10 folds.
# which.min() giver kolonnen (model) med mindst gennemsnitlige MSE.
# vi får altså den model der giver den bedste balance mellem bias og variance
best.size <- which.min(colMeans(cv.errors))
best_subset_cv_mse <- min(colMeans(cv.errors))
# regsbsets(...) laver best subset selection på alt træningsdata
# og bruger best.size antal kolonner
best.fit <- regsbsets(formula, data = train, nvmax = best.size)
# Funktion vi lavede tidligere
best.pred <- predict.regsbsets(best.fit, test, id = best.size, formula = formula)
# Vi udregner først residualerne, det er forskellen mellem
# de predictede og faktiske værdier
# Derefter kvadreret fejl ^2 for at finde MSE, findes gennemsnittet med mean()
# til sidst kvadratrods for "root mean squared error"
# Vi bruger RMSE fordi tallet er i samme enhed som tilskuerne og er let at forstå
best_subset_rmse <- sqrt(mean((best.pred - y_test)^2))
# Trækker de valgte predictors ud fra modellen for at vise dem i resultatet.
best_subset_coef <- coef(best.fit, best.size)

# ----- Ridge Regression -----
# model.matrix() beskrevet overfor.
# Vi bruger -1 subset for at fjerne Intercept rækken
x_train <- model.matrix(formula, train)[, -1]
x_test <- model.matrix(formula, test)[, -1]
# cv.glmnet(...) deler træningsdata i 10 folds, alpha = 0 vælger Ridge
# den laver ridge regression for mange værdier af regulariseringsstyrken lambda
# og tester hver model ved cross validation.
# Så finder den værdien af lambda der har lavest MSE
ridge.cv <- cv.glmnet(x_train, y_train, alpha = 0)
ridge_cv_mse <- min(ridge.cv$cvm)
ridge.pred <- predict(ridge.cv, newx = x_test, s = ridge.cv$lambda.min)
ridge_rmse <- sqrt(mean((ridge.pred - y_test)^2))
# coef() trækker koefficienterne ud så de vises i resultaterne
ridge_coef <- as.matrix(coef(ridge.cv, s = "lambda.min"))

# ----- Lasso Regression -----
lasso.cv <- cv.glmnet(x_train, y_train, alpha = 1)
lasso_cv_mse <- min(lasso.cv$cvm)
lasso.pred <- predict(lasso.cv, newx = x_test, s = lasso.cv$lambda.min)
lasso_rmse <- sqrt(mean((lasso.pred - y_test)^2))
lasso_coef <- as.matrix(coef(lasso.cv, s = "lambda.min"))

# ----- PCR -----
# Fit PCR-modellen på træningsdata, scale = TRUE skalerer alle variabler
# (PCR/PLS kræver standardisering)
# validation = "CV" udfører cross-validation for at vurdere
# hvor mange Principal Components der skal bruges
# PCR vælger Principal komponenter der forklarer X bedst
pcr.fit <- pcr(formula, data = train, scale = TRUE, validation = "CV")
# Maksimalt tilladte komponenter

```

```

# (kan ikke overstige antal rækker eller antal prædiktorer)
max_comp <- min(nrow(train) - 1, ncol(train) - 1)
# Hent CV-fejl (MSEP) for hvert komponent-antal
# MSEP(...) viser [statistic, response, component]
cv_msep <- MSEP(pcr.fit)$val[1,1,]
# which.min(cv_msep) finder antal komponenter med lavest MSEP
# min() sikrer at vi ikke går over max_comp
best_pcr_ncomp <- min(which.min(cv_msep), max_comp)
pcr_cv_mse <- min(cv_msep, na.rm = TRUE)
# Forudsig testdata med det optimale antal komponenter
pcr.pred <- predict(pcr.fit, newdata = test, ncomp = best_pcr_ncomp)
# Beregn RMSE på testdata
pcr_rmse <- sqrt(mean((pcr.pred - y_test)^2))

# ----- PLS -----
# PLS vælger Latente komponenter der forklarer Y bedst
# pls() fungerer på samme måde som pcr() men
# finder optimalt antal latent components i stedet.
pls.fit <- pls(formula, data = train, scale = TRUE, validation = "CV")

cv_msep_pls <- MSEP(pls.fit)$val[1,1,]
best_pls_ncomp <- min(which.min(cv_msep_pls), max_comp)
pls_cv_mse <- min(cv_msep_pls, na.rm = TRUE)
pls.pred <- predict(pls.fit, newdata = test, ncomp = best_pls_ncomp)
pls_rmse <- sqrt(mean((pls.pred - y_test)^2))

# ----- Returner resultater -----
list(
  RMSE = tibble(
    Model = c("Lineær (alle variabler)", "Best subset", "Ridge", "Lasso", "PCR", "PLS"),
    RMSE = c(lm_rmse, best_subset_rmse, ridge_rmse, lasso_rmse, pcr_rmse, pls_rmse)
  ),
  CV_RMSE = tibble(
    Model = c("Best subset", "Ridge", "Lasso", "PCR", "PLS"),
    CV_RMSE = sqrt(c(best_subset_cv_mse, ridge_cv_mse, lasso_cv_mse, pcr_cv_mse, pls_cv_mse))
  ),
  Linear = list(model = lm.fit),
  BestSubset = list(coef = best_subset_coef, size = best.size),
  Ridge = list(coef = ridge_coef, lambda_min = ridge.cv$lambda.min),
  Lasso = list(coef = lasso_coef, lambda_min = lasso.cv$lambda.min),
  PCR = list(ncomp = best_pcr_ncomp, model = pcr.fit),
  PLS = list(ncomp = best_pls_ncomp, model = pls.fit)
)
}

# Kør modeller på de udvalgte variabler
results_tilskuere <- run_all_models(vff_variabler, "tilskuere")

```

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

```
results_10 <- run_all_models(vff_10, "tilskuere")
```

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in = force.in, : 1 linear dependencies found

Reordering variables and trying again:

```
results_7 <- run_all_models(vff_7, "tilskuere")
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

```
Reordering variables and trying again:
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
results_3 <- run_all_models(vff_3, "tilskuere")
```

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

```
Warning in leaps.setup(x, y, wt = wt, nbest = nbest, nvmax = nvmax, force.in =  
force.in, : 1 linear dependencies found
```

Reordering variables and trying again:

results_tilskuere

\$RMSE

A tibble: 6 x 2

Model	RMSE
<chr>	<dbl>
1 Lineær (alle variabler)	1071.
2 Best subset	1352.
3 Ridge	1099.
4 Lasso	1058.
5 PCR	1223.
6 PLS	1075.

\$CV_RMSE

A tibble: 5 x 2

Model	CV_RMSE
<chr>	<dbl>
1 Best subset	1377.
2 Ridge	1350.
3 Lasso	1351.
4 PCR	1387.
5 PLS	1328.

\$Linear

\$Linear\$model

Call:

```
lm(formula = formula, data = train)
```

Coefficients:

(Intercept)	runde	år
-441636,4898	13,4118	222,5616
ugedag.L	ugedag.Q	ugedag.C
47,7991	470,5982	-582,3505
ugedag^4	ugedag^5	tidsgruppemidt
702,6361	291,2866	-35,8323
tidsgruppesent	seneste_kampaufgjort	seneste_kamptabt
-120,2052	-477,1282	-631,7960
vff_vundet_2	regn_gruppelidt regn	regn_gruppemeget regn
94,9770	10,5790	-661,5633
temperatur	vind	akk_indbyggertal
39,3531	-45,8778	-0,3824

kamp_gruppemiddel	kamp_gruppestor	ferie_flagja
361,0882	2140,1628	-470,6038

\$BestSubset

\$BestSubset\$coef

(Intercept)	år	ugedag.Q	ugedag.C
-463707,271258	234,497962	389,704797	-277,661385
ugedag^5	seneste_kamptabt	vff_vundet_2	temperatur
422,308911	-343,947322	277,124005	17,934229
vind	akk_indbyggertal	kamp_gruppemiddel	ferie_flagja
-84,492355	-0,381537	-1147,035211	-344,768519
ugedag^6			
327,297316			

\$BestSubset\$size

[1] 12

\$Ridge

\$Ridge\$coef

	lambda.min
(Intercept)	-183592,41779744
runde	11,90338779
år	93,60071544
ugedag.L	-31,82541930
ugedag.Q	604,71170588
ugedag.C	-215,62957338
ugedag^4	71,24418936
ugedag^5	513,76632703
ugedag^6	147,86891273
tidsgruppemidt	-121,04529558
tidsgruppesent	-16,11884484
seneste_kampaufgjort	-451,09288897
seneste_kamptabt	-589,00948689
vff_vundet_2	254,74793853
regn_gruppelidt regn	-124,98293349
regn_gruppemeget regn	-760,28415727
temperatur	40,34559781
vind	-48,27141597
akk_indbyggertal	-0,09302844
kamp_gruppemiddel	207,64979161
kamp_gruppestor	1872,36157851
ferie_flagja	-332,14704370

\$Ridge\$lambda_min

[1] 75,38484

\$Lasso

\$Lasso\$coef

	lambda.min
(Intercept)	-266036,4793884
runde	10,5180385
år	134,8171773

```

ugedag.L          -54,3601825
ugedag.Q          468,2931449
ugedag.C          -90,9845658
ugedag^4          0,0000000
ugedag^5          505,9265194
ugedag^6          0,0000000
tidsgruppemidt    -110,7116774
tidsgruppessent    0,0000000
seneste_kampaufgjort -430,8757838
seneste_kamptabt   -583,8763400
vff_vundet_2      159,6922462
regn_gruppelidt regn -47,8124840
regn_gruppemeget regn -646,8977731
temperatur        36,4378217
vind              -43,5250691
akk_indbyggertal  -0,1770114
kamp_gruppemiddel 211,4676927
kamp_gruppestor   1951,4968199
ferie_flagja      -329,2882858

```

```

$Lasso$lambda_min
[1] 18,2441

```

```

$PCR
$PCR$ncomp
[1] 12

```

```

$PCR$model
Principal component regression, fitted with the singular value decomposition algorithm.
Cross-validated using 10 random segments.
Call:
pcr(formula = formula, data = train, scale = TRUE, validation = "CV")

```

```

$PLS
$PLS$ncomp
[1] 11

```

```

$PLS$model
Partial least squares regression, fitted with the kernel algorithm.
Cross-validated using 10 random segments.
Call:
plsrf(formula = formula, data = train, scale = TRUE, validation = "CV")

```

results_10

```

$RMSE
# A tibble: 6 x 2
  Model          RMSE
  <chr>         <dbl>
1 Lineær (alle variabler) 641.
2 Best subset          627.
3 Ridge                628.
4 Lasso                602.

```

```
5 PCR          683.
6 PLS          638.
```

```
$CV_RMSE
```

```
# A tibble: 5 x 2
```

```
  Model      CV_RMSE
  <chr>      <dbl>
1 Best subset  493.
2 Ridge       495.
3 Lasso       475.
4 PCR        489.
5 PLS        484.
```

```
$Linear
```

```
$Linear$model
```

```
Call:
```

```
lm(formula = formula, data = train)
```

```
Coefficients:
```

```
      (Intercept)      salg_10      runde
      -107753,72700      1,81376      -3,26175
      år      ugedag.L      ugedag.Q
      54,23429      -227,51607      279,25345
      ugedag.C      ugedag^4      ugedag^5
      -123,72038      162,23963      512,72561
      tidsgruppemidt      tidsgruppesent      seneste_kampaufgjort
      284,47302      -254,28602      -168,87269
      seneste_kamptabt      vff_vundet_2      akk_indbyggertal
      -178,39237      -37,00335      -0,08849
      kamp_gruppemiddel      kamp_gruppestor      ferie_flagja
      13,33390      520,47696      -225,60048
```

```
$BestSubset
```

```
$BestSubset$coef
```

```
      (Intercept)      salg_10      år      ugedag.Q
      -25660,360422      1,952152      13,050560      256,012556
      tidsgruppemidt      tidsgruppesent      seneste_kamptabt      vff_vundet_2
      307,543176      116,368657      -81,976093      38,188747
      kamp_gruppemiddel      ferie_flagja
      -253,911546      -85,066242
```

```
$BestSubset$size
```

```
[1] 9
```

```
$Ridge
```

```
$Ridge$coef
```

```
      lambda.min
      (Intercept)      -39668,689069929
      salg_10      1,610946707
      runde      -2,018993817
      år      20,321444320
      ugedag.L      -94,863009926
```

ugedag.Q	221,887580016
ugedag.C	32,723778694
ugedag^4	125,792811237
ugedag^5	72,458020166
ugedag^6	494,514126772
tidsgruppemidt	245,624321325
tidsgruppesent	-78,359813327
seneste_kampuafgjort	-126,448741486
seneste_kamptabt	-195,585904003
vff_vundet_2	23,633642286
akk_indbyggertal	-0,004798031
kamp_gruppemiddel	19,779149560
kamp_gruppestor	618,707994668
ferie_flagja	-184,691631500

```
$Ridge$lambda_min
[1] 158,3439
```

```
$Lasso
$Lasso$coef
```

	lambda.min
(Intercept)	-23772,9463574
salg_10	1,8400861
runde	-0,7799175
år	12,1466691
ugedag.L	0,0000000
ugedag.Q	173,9352550
ugedag.C	0,0000000
ugedag^4	156,5872814
ugedag^5	0,0000000
ugedag^6	269,6798349
tidsgruppemidt	239,7231030
tidsgruppesent	0,0000000
seneste_kampuafgjort	-92,1042092
seneste_kamptabt	-116,5114137
vff_vundet_2	0,0000000
akk_indbyggertal	0,0000000
kamp_gruppemiddel	0,0000000
kamp_gruppestor	432,4889781
ferie_flagja	-101,3778368

```
$Lasso$lambda_min
[1] 12,54848
```

```
$PCR
$PCR$ncomp
[1] 10
```

```
$PCR$model
```

Principal component regression, fitted with the singular value decomposition algorithm.
Cross-validated using 10 random segments.

Call:

```
pcr(formula = formula, data = train, scale = TRUE, validation = "CV")
```

```

$PLS
$PLS$ncomp
[1] 10

$PLS$model
Partial least squares regression, fitted with the kernel algorithm.
Cross-validated using 10 random segments.
Call:
plsrf(formula = formula, data = train, scale = TRUE, validation = "CV")

```

results_7

```

$RMSE
# A tibble: 6 x 2
  Model          RMSE
  <chr>        <dbl>
1 Lineær (alle variabler) 383.
2 Best subset          340.
3 Ridge                425.
4 Lasso                343.
5 PCR                  548.
6 PLS                  379.

```

```

$CV_RMSE
# A tibble: 5 x 2
  Model      CV_RMSE
  <chr>    <dbl>
1 Best subset  297.
2 Ridge       330.
3 Lasso       291.
4 PCR         305.
5 PLS         304.

```

```

$Linear
$Linear$model

```

```

Call:
lm(formula = formula, data = train)

```

```

Coefficients:
      (Intercept)      salg_10      salg_7
-55283,98366      1,69021      0,95744
      runde      år      ugedag.L
-4,21383      27,75560     -135,61643
      ugedag.Q      ugedag.C      ugedag^4
109,44819      71,98532     -63,07960
      ugedag^5      tidsgruppemidt      tidsgruppesent
206,49510      99,19123     -115,18048
seneste_kampaufgjort      seneste_kamptabt      vff_vundet_2
2,72682      -18,43258      48,62757
      akk_indbyggertal      kamp_gruppemiddel      kamp_gruppestor
-0,03641      -1,92544      234,01636
      ferie_flagja
-97,68484

```

```
$BestSubset
$BestSubset$coef
  (Intercept)      salg_10      salg_7      år  ferie_flagja
-12926,161476    1,765502    1,059542    6,478537   -38,037144
```

```
$BestSubset$size
[1] 4
```

```
$Ridge
$Ridge$coef
              lambda.min
(Intercept)  -29182,320305179
salg_10       1,478720072
salg_7        0,951353461
runde        -2,888210445
år            14,846431586
ugedag.L     -21,767158620
ugedag.Q      79,832685765
ugedag.C     115,676909132
ugedag^4      56,872471749
ugedag^5    -109,341665884
ugedag^6     328,594110524
tidsgruppemidt 86,416630027
tidsgruppesent -19,762967554
seneste_kampaufgjort 2,199014996
seneste_kamptabt -70,862585563
vff_vundet_2   69,212876758
akk_indbyggertal 0,001734739
kamp_gruppemiddel 31,237036285
kamp_gruppestor 395,079021701
ferie_flagja   -90,946192308
```

```
$Ridge$lambda_min
[1] 158,3439
```

```
$Lasso
$Lasso$coef
              lambda.min
(Intercept)  -10734,6313880
salg_10       1,7100699
salg_7        0,9658553
runde        0,0000000
år            5,4639824
ugedag.L     0,0000000
ugedag.Q      0,0000000
ugedag.C      4,6182374
ugedag^4      0,0000000
ugedag^5      0,0000000
ugedag^6      0,0000000
tidsgruppemidt 26,5961014
tidsgruppesent 0,0000000
```

```

seneste_kampaufgjort      0,0000000
seneste_kamptabt          0,0000000
vff_vundet_2              0,0000000
akk_indbyggertal          0,0000000
kamp_gruppemiddel         0,0000000
kamp_gruppestor           142,9999846
ferie_flagja              0,0000000

```

```

$Lasso$lambda_min
[1] 31,81497

```

```

$PCR
$PCR$ncomp
[1] 11

```

```

$PCR$model
Principal component regression, fitted with the singular value decomposition algorithm.
Cross-validated using 10 random segments.
Call:
pcr(formula = formula, data = train, scale = TRUE, validation = "CV")

```

```

$PLS
$PLS$ncomp
[1] 11

```

```

$PLS$model
Partial least squares regression, fitted with the kernel algorithm.
Cross-validated using 10 random segments.
Call:
plsr(formula = formula, data = train, scale = TRUE, validation = "CV")

```

results_3

```

$RMSE
# A tibble: 6 x 2
  Model      RMSE
  <chr>    <dbl>
1 Lineær (alle variabler) 182.
2 Best subset           169.
3 Ridge                 212.
4 Lasso                 174.
5 PCR                   299.
6 PLS                   182.

```

```

$CV_RMSE
# A tibble: 5 x 2
  Model      CV_RMSE
  <chr>    <dbl>
1 Best subset 167.
2 Ridge      203.
3 Lasso      160.
4 PCR        175.
5 PLS        167.

```

```
$Linear
$Linear$model
```

```
Call:
lm(formula = formula, data = train)
```

```
Coefficients:
      (Intercept)      salg_10      salg_7
    -28313,14076      1,34042      1,03994
      salg_3      runde      år
      0,77403     -0,53205     14,22403
    ugedag.L    ugedag.Q    ugedag.C
    -29,70137     17,92032    -92,66373
    ugedag^4    ugedag^5    tidsgruppemidt
    -52,29552     156,63448     -66,51902
  tidsgruppesent  seneste_kampuafgjort  seneste_kamptabt
    -91,72348     -20,66904      8,78573
    vff_vundet_2  akk_indbyggertal  kamp_gruppemiddel
      5,64149      -0,01893     -5,85155
  kamp_gruppestor    ferie_flagja
    157,34399     -37,96785
```

```
$BestSubset
$BestSubset$coef
      (Intercept)      salg_10      salg_7      salg_3      år
    -6920,9058643      1,3748944      1,0774223      0,7796401      3,4699334
      ugedag^4  tidsgruppesent  ferie_flagja
    -40,4046644      38,3985736      13,0630345
```

```
$BestSubset$size
[1] 7
```

```
$Ridge
$Ridge$coef
      lambda.min
  (Intercept) -17266,7360862114
    salg_10      1,1345642910
    salg_7      0,9985204830
    salg_3      0,8699954945
    runde      0,6978251037
    år      8,8219518876
    ugedag.L      7,6239130327
    ugedag.Q     32,3881544403
    ugedag.C     -5,6609442686
    ugedag^4    -79,6839061482
    ugedag^5     -7,0465593161
    ugedag^6    235,5164141396
  tidsgruppemidt -61,5658951377
  tidsgruppesent -10,7340045351
  seneste_kampuafgjort -28,2246415592
  seneste_kamptabt -32,0797589668
  vff_vundet_2    16,7200538919
```

akk_indbyggertal	0,0001234724
kamp_gruppemiddel	22,6605158043
kamp_gruppestor	287,4936846877
ferie_flagja	-35,0052526608

\$Ridge\$lambda_min
[1] 158,3439

\$Lasso

\$Lasso\$coef

	lambda.min
(Intercept)	-8635,7064797
salg_10	1,3563700
salg_7	1,0311606
salg_3	0,7496625
runde	0,0000000
år	4,3675309
ugedag.L	0,0000000
ugedag.Q	0,0000000
ugedag.C	-12,4475773
ugedag^4	0,0000000
ugedag^5	0,0000000
ugedag^6	37,7060007
tidsgruppemidt	-12,0559007
tidsgruppesent	0,0000000
seneste_kampaufgjort	-2,5113807
seneste_kamptabt	0,0000000
vff_vundet_2	0,0000000
akk_indbyggertal	0,0000000
kamp_gruppemiddel	0,0000000
kamp_gruppestor	110,5387110
ferie_flagja	0,0000000

\$Lasso\$lambda_min
[1] 8,649182

\$PCR

\$PCR\$ncomp

[1] 12

\$PCR\$model

Principal component regression, fitted with the singular value decomposition algorithm.

Cross-validated using 10 random segments.

Call:

pcr(formula = formula, data = train, scale = TRUE, validation = "CV")

\$PLS

\$PLS\$ncomp

[1] 10

\$PLS\$model

Partial least squares regression, fitted with the kernel algorithm.

Cross-validated using 10 random segments.

```
Call:
plsr(formula = formula, data = train, scale = TRUE, validation = "CV")
```

C.1 Sammenlign på validation de forskellige MSE fra hver model

Sammenligning på RMSE på modellerne i results_tilskuere, som er uden billetsalg.

Den lineære funktion har test RMSE på 1164 Best subset selection - Test RMSE på 1418 Lasso - Test RMSE på 1176 Ridge regression - Test RMSE på 1171 PCR - Test RMSE på 1291 PLS - Test RMSE på 1170

Den lineære regression klarer sig bedst. Men Ridge, Lasso og PLS ligger også tæt på. Med en anden seed kunne de godt bytte plads.

Lasso og Ridge skrumper nogle af koefficienterne, men det ligner at der ikke er nogen stærk multikollinearitet eller unødvendige variabler.

Best subset selection har en højere test_RMSE på 1418, Det kan muligvis tyde på overfitting da vi ikke har tilstrækkeligt nok data.

PCR klarer sig heller ikke så godt - det kan tyde på overfitting

results_10, results_7, results_3 RMSE værdierne falder gradvist når vi tilføjer de stærke predictors som er billetsalg. Alle modellerne ligger tæt på hinanden, men PCR bliver lidt højere end de andre.

D Evaluering

Vi har kun 194 rækker data til vores model som forudser tilskuertallet flere måneder forude, og vi har kun 209 rækker data til modellerne som forudser 10 dage, 7 dage og 3 dage før.

Siden datamængden er lille, får vi højere varians. En outlier i test datasættet kan få test RMSE til at flyve i vejret i sammenligning med CV RMSE. I modsætning til det, så hvis test datasættet er lettere end træningsdatasættet bliver test RMSE lavere end CV RMSE.

Vi vælger den lineære model til at predict worst case og best case, da den er simplest og har sammen med PLS og Lasso den laveste test_RMSE.

D.1 Hvordan er Test-RMSE sammenlignet med CV-RMSE

results_tilskuere: Best subset selection - Test RMSE på 1418 og CV RMSE på 1434. De ligger ret tæt på hinanden Ridge regression - Test RMSE på 1171 og CV RMSE på 1306. Test RMSE ligger tydeligt lavere, det kunne skyldes at vi ikke har ret meget data at teste på, og det giver en lidt heldig split som passer modellen bedre. Lasso - Test RMSE på 1176 og CV RMSE på 1282. Minder om Ridge tilfældet PCR - Test RMSE på 1291 og CV RMSE på 1344. Tæt på hinanden, test lidt lavere PLS - Test RMSE på 1170 og CV RMSE på 1329. Minder også om Ridge tilfældet

CV_RMSE ligger generelt lidt højere end test RMSE - det kan tyde på test sættet er lidt lettere end træningsdatasættet. Vi tænker det skyldes en tilfældighed pga. en lille sample size.

results_10, results_7, results_3 Når vi inkluderer billetsalg fra op til 10 dage før kampen, falder fejlene mere og mere. Best subset selection, Lasso og PLS klarer sig bedst og man kan vurdere billetsalg er en stærk predictor. Ridge ligger lidt højere end de andre på test_RMSE. Dette sker fordi Ridge laver alle predictors tættere på 0, også den stærke billetsalg predictor. PCR bliver ustabil i test_RMSE - det kan tyde på overfitting

D.2 Fortokning

D.3 Prædiktions på nye data (fx best-case/worst-case scenario for de første kampe i foråret)

```
### Prædiktions på nye data (fx best-case/worst-case scenario for de første kampe i foråret)
# lineær model
lm.fit <- results_tilskuere$Linear$model

# --- BEST CASE scenario --- VFF-BIF spiller søndag d. 15/2 i runde 20, hjemmekamp
best_case <- tibble(
  runde = 20,
  år = 2026,
  ugedag = factor("Søn", levels = levels(vff_variabler$ugedag)),
  tidsgruppe = factor("sent", levels = levels(vff_variabler$tidsgruppe)),
  seneste_kamp = factor("vundet", levels = levels(vff_variabler$seneste_kamp)),
  vff_vundet_2 = 1,
  regn_gruppe = factor("ingen regn", levels = levels(vff_variabler$regn_gruppe)),
  temperatur = 8, # forårsvej
  vind = 3, # mild vind
  akk_indbyggertal = 10246,
  kamp_gruppe = factor("stor", levels = levels(vff_variabler$kamp_gruppe)),
  ferie_flag = factor("ja", levels = levels(vff_variabler$ferie_flag))
)

# --- WORST CASE scenario ---
worst_case <- tibble(
  runde = 20,
  år = 2026,
  ugedag = factor("Søn", levels = levels(vff_variabler$ugedag)),
  tidsgruppe = factor("sent", levels = levels(vff_variabler$tidsgruppe)),
  seneste_kamp = factor("tabt", levels = levels(vff_variabler$seneste_kamp)),
  vff_vundet_2 = 0,
  regn_gruppe = factor("meget regn", levels = levels(vff_variabler$regn_gruppe)),
  temperatur = -5, # koldt
  vind = 20, # kraftig vind
  akk_indbyggertal = 10246,
  kamp_gruppe = factor("stor", levels = levels(vff_variabler$kamp_gruppe)),
  ferie_flag = factor("ja", levels = levels(vff_variabler$ferie_flag))
)

# --- 3) Predict med 95% prediktionsinterval (point estimate + interval for nye observationer) ---
pred_best <- predict(lm.fit, newdata = best_case, interval = "prediction", level = 0.95)
pred_worst <- predict(lm.fit, newdata = worst_case, interval = "prediction", level = 0.95)

# Saml resultater
results <- tibble(
  Scenario = c("Best case", "Worst case"),
  Fit = c(pred_best[, "fit"], pred_worst[, "fit"]),
  PI_lower = c(pred_best[, "lwr"], pred_worst[, "lwr"]),
  PI_upper = c(pred_best[, "upr"], pred_worst[, "upr"])
)

print(results)
```

```
# A tibble: 2 x 4
  Scenario    Fit PI_lower PI_upper
  <chr>      <dbl>    <dbl>    <dbl>
1 Best case 7665.    4757.    10573.
2 Worst case 4985.    1618.     8352.
```

D.4 Sammenlign resultatet med formålet med undersøgelsen.

Vi kan se i best case at når vejret er godt og når VFF har vundet de sidste to kampe osv. så kommer der flere tilskuere til kampen. Det stemmer overens med formålet for at lave machine learning modellen. Den giver et data-drevet billede af hvor mange der kommer til kampen og kan være værdiskabende hvis implementeret korrekt.

E Eksempler på markdown-sproget

Her hentes et billede

Sådan henviser/refererer du til en figur: [1](#)



Figur 1: Faserne i CRISP-DM.

Her er et eksempel på anvendelse af en png fil, som skal ligge i dit working directory. Tekst mellem `[]` er figurteksten, som du ønsker i din opgave. Tekst mellem `()` er (placering og) navn på figuren. Tekst mellem `{}` er et henvisningsnavn, som du kan bruge til at henvise til figuren i din tekst.

Her er et eksempel på en figur, som jeg henviser til [2](#). En figur som JEG har lavet længere nede i koden.

Her et eksempel med en tabel [1](#), som JEG har lavet længere nede.

Her er noget tekst blabla, og nu vil jeg gerne lave en reference til afsnit [1](#).

Reference fx fra interviews bla.bla..bla... (VFF 2025) eller en bog (Bang 2024) osv

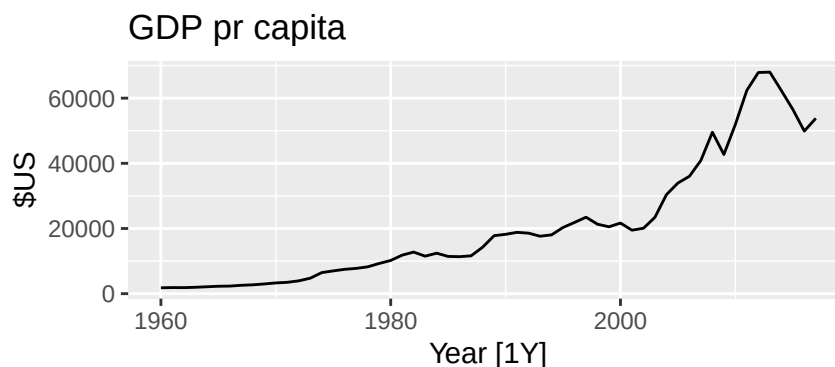
Reference til en ligning: blabla [1](#) blabla

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^k y_{t+j} \quad (1)$$

Jeg kan også referere til fx en værdi, som jeg har udregnet i en kode chunk:

I forbindelse med den simple lineære regression er $R^2 = 0,59$

Lav gerne mange kode chunks i stedet for at have lange chunks - det giver et bedre overblik. Indholdet i en kode chunk skal være sammenhængende. Men lav gerne flere i forbindelse med fx, at du laver en Lasso regression.



Figur 2: Australisk BNP pr capita

Tabel 1: Outliers og indflydelsesrige observationer.

Observation	Outlier	Høj leverage	Indflydelsesrig observation	Reference til plot
16	Ja	Nej	Nej	Residuals vs Fitted, Q-Q Plot, Scale-Location, Boxplot
17	Ja	Nej	Nej	Residuals vs Fitted, Q-Q Plot, Scale-Location
39	Nej	Ja	Nej	Residuals vs Leverage
117	Nej	Ja	Ja	Residuals vs Leverage (Cook's Distance)

F Fed og kursiv, og punktform

- **Daniel** Daniel skreved med fed skrift
 - *Olga* Olga skrevet i kursiv
 - * ***Palle*** Palle skrevet med fed skrift og i kursiv

“-” angiver punktform - bemærk de forskellige niveauer